FROM INCIDENTS TO REPLAYABLE EXPLOITS: AN EMPIRICAL STUDY OF DEFI ATTACK SURFACES

ABSTRACT

Public DeFi hack reporting is abundant, but research claims about exploitability often collapse together corpus statistics, protocol-specific reasoning, and proof-of-concept replay results. This paper rebuilds that pipeline explicitly. Using a normalized corpus of 2,036 public incidents, including 1,378 DeFi-labeled cases, we combine three evidence layers already present in the repository: a broad incident corpus, five deep protocol audits with 87 manually triaged findings, and a 24-protocol pure on-chain verification study augmented by replay engineering for representative historical exploits. The resulting picture is narrower than raw incident volume implies. In the verified protocol set, only one protocol exhibits a high-confidence pure on-chain exploit path, while three additional cases are conditional or integration-dependent, yielding an exposure ratio of approximately 16.7%. The replay batch further shows that only 3 of 12 shortlisted incidents have been converted into concrete replay logic so far, for a replay conversion rate of 25%. These results support a methodological claim: DeFi security research should be framed as an incident-to-replay workflow, not as a flat list of hacks or scanner findings. We conclude with a conservative but actionable interpretation of which exploit families remain most replayable and what evidence is still missing for stronger claims.

1 INTRODUCTION

Decentralized finance has generated a large public archive of incidents, exploit writeups, protocol post-mortems, and security dashboards, yet those artifacts rarely line up into a single evidentiary chain. Corpus-wide datasets are broad but noisy; protocol audits are deep but local; exploit proofs of concept are concrete but narrow. The practical effect is that discussions of DeFi security often drift between incompatible units of analysis. A repository may contain hundreds of scanner findings, a dashboard may report thousands of incidents, and a post-mortem may present a single elegant exploit path, but none of those views alone answers a core research question: which publicly reported incidents correspond to replayable pure on-chain exploit mechanisms rather than operational compromise, social engineering, or context-specific failures?

This paper addresses that gap using the artifact set that already exists in the repository. The repository contains a normalized incident corpus derived from the SlowMist archive, five deep audits of high-value protocols, a 24-protocol study of pure on-chain attack surfaces, and a growing replay suite for historically important incidents. Instead of treating these assets as separate workstreams, we assemble them into a staged workflow inspired by research automation systems such as ARA. The goal is not to claim that every incident has been reproduced. The goal is to show how broad incident evidence can be progressively filtered into mechanism-level claims with explicit confidence boundaries.

Our empirical thesis is conservative. Public DeFi incident volume overstates replayable exploitability for mature protocols. The reason is not that the incident feeds are wrong, but that they aggregate distinct phenomena: direct contract exploits, flash-loan-assisted logic failures, oracle manipulation, governance abuse, bridge-message abuse, private-key compromise, wallet theft, scams, and operational leakage. Once those classes are normalized and compared against protocol-level verification, the set of high-confidence pure on-chain attack paths becomes much smaller. Within the current repository artifacts, oracle manipulation, flash-loan-assisted logic abuse, and callback-driven reen-

trancy remain the most replayable classes, whereas many other reported incidents either depend on off-chain compromise or remain only conditionally exploitable.

The paper makes three contributions. First, it presents an explicit incident-to-replay workflow that links corpus construction, protocol screening, replay engineering, and paper synthesis. Second, it provides a quantitative summary of the repository evidence: 2,036 incidents, 1,378 DeFi-labeled cases, 87 manually triaged audit findings across five high-value protocols, and a 24-protocol pure on-chain verification study. Third, it distills these materials into a set of claims that are stronger than a scanner report but more honest than an overconfident exploit catalog. The result is not the final word on DeFi exploitability, but it is a reproducible starting point for a more disciplined research program.

2 Related Work

Prior DeFi security research has established the basic threat landscape but often stops short of an artifact-level bridge between incident archives and replay validation. Early work on flash-loan-driven exploitation demonstrated how composability, low-friction borrowing, and oracle dependence can transform latent logic mistakes into capital-efficient attacks (Qin et al., 2021). Systems work on decentralized exchanges and transaction ordering showed that adversarial execution in DeFi must be understood at the level of transaction sequencing and composability rather than isolated contracts (Daian et al., 2020). Broader overviews of DeFi and its security assumptions described the ecosystem as a layered stack of protocols, incentives, and integration risks, highlighting the methodological difficulty of attributing losses to a single locus of failure (Werner et al., 2021; Zhou et al., 2023).

A second body of work comes from protocol audits and vendor-style security reviews. These analyses are useful because they identify concrete code-level weaknesses, such as access-control gaps, arithmetic precision loss, or unsafe external-call structure. However, audit artifacts also have a known limitation: they mix true exploit preconditions with hardening advice and context-dependent design concerns. A finding that is valid at the code level may never become a replayable exploit under realistic liquidity, oracle, or governance assumptions. Our own repository reflects this tension. The scanner produced 651 findings across five protocols, but manual triage reduced the set to 87 unique issues, illustrating how a research paper built directly on raw scanner output would exaggerate exploitability.

A third line of evidence comes from public incident databases and post-mortems. Resources such as the SlowMist hacked archive and protocol-authored incident analyses are indispensable for historical coverage and case reconstruction (SlowMist, 2026; Euler Labs, 2023; The Block, 2023; ZDNet, 2020). Yet those sources are heterogeneous. They vary in how they define the target, classify the root cause, estimate loss, and separate direct protocol exploits from ancillary compromise. Our approach uses those archives as the first stage of research, not the last. The paper therefore occupies a middle ground: broader than a single incident post-mortem, narrower than a generic ecosystem survey, and more verification-oriented than a standalone audit report.

This positioning is important for evaluation. A survey paper is judged by coverage, an audit report is judged by depth on a specific codebase, and a post-mortem is judged by the accuracy of a single causal reconstruction. Our paper is judged by the coherence of the transition between these artifact classes. The intended contribution is therefore methodological: we show how a repository can expose a disciplined progression from noisy incident evidence, to protocol-level exploitability claims, to replay-ready exploit engineering. To our knowledge, this is the missing layer in much of the existing DeFi literature, which tends to stop at either ecosystem-wide taxonomy or protocol-local analysis without specifying how one becomes the other.

3 Method

3.1 Five-Stage Workflow

The refactored paper module adopts a five-stage workflow adapted from the design pattern used in ARA: idea formation, experiment derivation, writing, revision, and build export. In this repository, those stages map to concrete directories and artifacts rather than abstract prompts. The idea stage

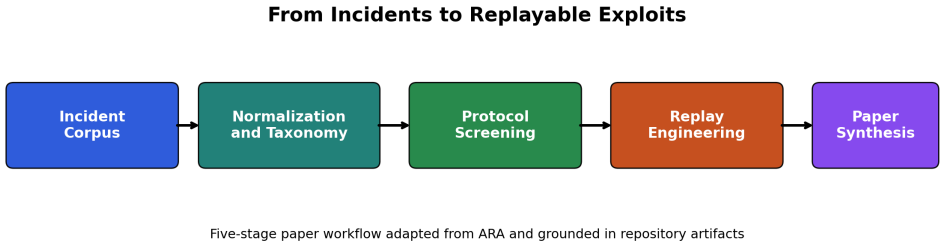

Figure 1: Refactored paper workflow. The pipeline begins with a broad incident corpus, narrows through normalization and protocol screening, and only then promotes selected cases into replay engineering and paper claims.

Table 1: Evidence layers and the kinds of claims they support.

Layer	Primary artifacts	Supported claims
Incident corpus	Normalized incident CSV, yearly counts, family taxonomy, loss summaries	Broad ecosystem trends, dominant categories, shortlist construction, evidence that public feeds are heterogeneous
Protocol evidence	Deep audits, triaged findings, 24-protocol verification report	High-confidence statements about pure on-chain exploitability under a specific threat model
Replay evidence	Incident cards, replay tests, fork anchors, encoded exploit logic	Mechanism-level claims about exploit preconditions and reproduction feasibility

defines research questions and scope constraints. The experiment stage generates structured CSV summaries and figures from the normalized incident corpus and from protocol-study constants already supported by repository reports. The writing stage contains a blueprint, bibliography, modular LaTeX sections, and figure assets. The revision stage records weaknesses in the previous draft and the changes introduced in this refactor. Figure 1 shows the corresponding research workflow that connects incident collection to paper synthesis.

3.2 Evidence Layers

The workflow is built around three evidence layers. The first is a normalized incident dataset loaded from ‘data/processed/incidents_normalized_latest.csv‘, which currently contains 2,036 rows and 2,035 unique incident identifiers, of which 1,378 are labeled as DeFi. The second is a protocol evidence layer composed of five deep audits and a 24-protocol pure on-chain verification study. The third is a replay layer consisting of twelve shortlisted incidents, with concrete replay logic already encoded for Euler Finance, BonqDAO and AllianceBlock, and Lendf.Me.

We intentionally distinguish between *confirmed exploitability*, *conditional exploitability*, and *replay engineering progress*. Confirmed exploitability means that the repository contains high-confidence evidence of a pure on-chain attack path under the study’s threat model. Conditional exploitability means that the issue depends on downstream integrations, special market states, or missing runtime confirmation. Replay engineering progress is weaker still: it indicates whether the exploit path has been encoded as executable logic in tests, regardless of whether archive RPC validation has been completed.

Table 1 summarizes the paper’s central discipline: every claim must be attached to the layer that actually supports it. This prevents a common failure mode in security writing, where scanner findings

are discussed as if they were reproduced exploits or where incident counts are discussed as if they were code-level proof.

3.3 Scoring and Derived Metrics

Replay work is prioritized using a pragmatic scoring rule rather than a claim of theoretical severity. For an incident i, we define a replayability prioritization score as

$$S(i) = \alpha_f F(i) + \alpha_l \log(1 + L(i)) + \alpha_r R(i) + \alpha_c C(i),$$

where $F(i)$ weights historically replayable families such as oracle manipulation, flash-loan-assisted logic abuse, and reentrancy; $L(i)$ is the reported loss in U.S. dollars; $R(i)$ indicates whether a public exploit path or post-mortem exists; and $C(i)$ represents chain and contract identifiability. This score is used operationally for shortlist construction, not as a normative security metric.

For the paper, two aggregate metrics are more important. The first is replay conversion,

$$\text{ReplayConversion} = \frac{N_{\text{replay logic}}}{N_{\text{shortlisted}}},$$

which measures how many shortlisted incidents have already been turned into concrete replay logic. The second is protocol exposure ratio,

$$\text{ExposureRatio} = \frac{N_{\text{conditional}} + N_{\text{confirmed}}}{N_{\text{verified}}},$$

which measures how much of the verified protocol set remains exposed to either direct or integration-dependent pure on-chain attack paths. In the current artifact set, these values are $3/12 = 0.25$ and $4/24 \approx 0.167$, respectively.

3.4 Threat Model and Confidence Boundaries

The protocol-study layer uses a deliberately narrow threat model. An attack is treated as pure on-chain only if it can be executed in a single transaction or a short block window without relying on external compromise, extraordinary governance capture, or unrealistic market conditions. This choice biases the study toward precision rather than recall. It undercounts some real incidents, but it avoids conflating every observed loss with directly replayable contract exploitability. The replay layer inherits the same discipline: a scaffold or encoded exploit path is not described as fully verified unless archive RPC execution has actually succeeded.

4 Experiments and Results

4.1 Corpus-Level Results

The normalized corpus provides the broadest evidentiary layer in the repository. Table 2 summarizes the headline numbers. Two observations matter immediately. First, the corpus is large enough to support trend and taxonomy analysis rather than anecdotal reasoning. Second, it remains heterogeneous even after normalization. Approximately two-thirds of all incidents are labeled as DeFi, which means a non-trivial fraction of the broader archive still falls outside the paper's focus and must be filtered out before any exploitability claim is made.

Figure 2 shows the yearly count of DeFi-labeled incidents in the normalized corpus. The curve rises steeply through 2021, peaks in the 2022–2023 period, and then declines in the partial 2025–2026 window. This trend should not be read as a direct measure of protocol insecurity. It also reflects reporting intensity, ecosystem expansion, and dataset timing. Still, it establishes that the corpus is large enough to justify a workflow that prioritizes filtering and mechanism identification over one-off exploit storytelling.

Attack-family counts tell a more useful story than yearly volume. Figure 3 shows that the normalized DeFi subset is dominated by the `other` bucket, followed by contract bugs, flash-loan incidents, account compromise, bridge-message abuse, social engineering, and oracle manipulation. The size of the `other` bucket is itself an important result: public archives remain semantically noisy even

Table 2: Dataset and study summary derived from repository artifacts.

Metric	Value
Total normalized incidents	2,036
Unique incident identifiers	2,035
DeFi-labeled incidents	1,378
DeFi share of full corpus	67.68%
Loss parsing coverage within DeFi subset	68.36%
Deep-audited protocols	5
Protocols in pure on-chain verification study	24
Shortlisted replay incidents	12
Replay cases with concrete logic	3

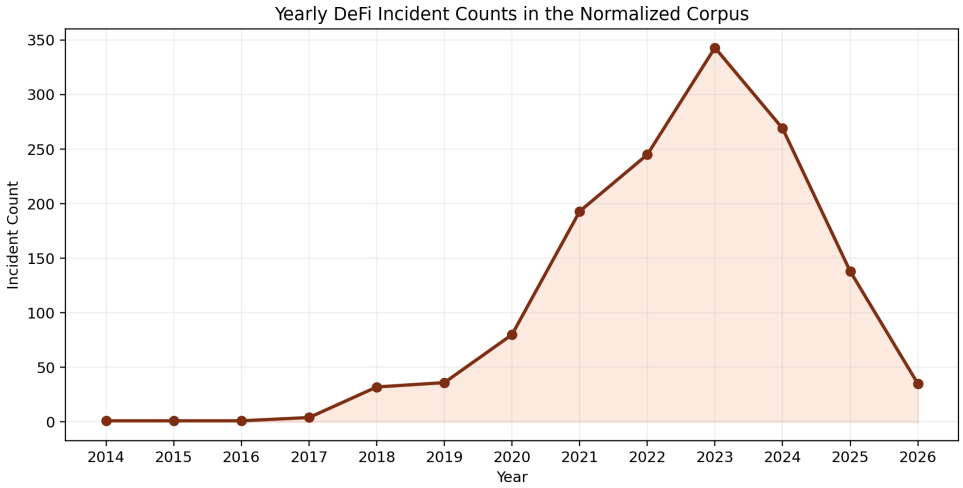

Figure 2: Yearly DeFi incident counts in the normalized corpus. The trend motivates broad corpus construction but does not by itself imply replayable exploitability.

after normalization, and any paper that translates those counts directly into exploit conclusions will overstate the prevalence of replayable smart-contract failures.

Loss statistics reinforce the same point. Within the DeFi-labeled subset, the median parsed loss is approximately $891,000, while the top twenty incidents account for roughly $17.26B in aggregate loss. Yet the appendix shows that many of the largest entries in the dataset fall into broad categories such as `other` or `supply_chain`. This is precisely why the paper separates incident magnitude from replayability. A high-loss event is important evidence for ecosystem risk, but it is not automatically evidence of a replayable pure on-chain contract exploit.

4.2 Protocol Evidence

The repository contains two complementary protocol-level datasets. The first consists of five deep audits on Lido, Aave, Uniswap, Curve, and Compound. The second is a 24-protocol pure on-chain verification study that asks a narrower question: which major protocols still expose single-transaction or short-window exploitability under a strict threat model? Figure 4 summarizes the manually triaged findings from the audit layer. The 87 findings include 5 critical, 16 high, 42 medium, and 24 low or informational issues. This distribution is useful because it demonstrates that code-level weakness density and replayable exploitability are not identical. Mature protocols still expose many hardening opportunities even when direct pure on-chain exploit paths are scarce.

The 24-protocol verification study yields an even sharper conclusion. As shown in Figure 5, 20 protocols were judged safe under the study's pure on-chain threat model, 3 were conditional or

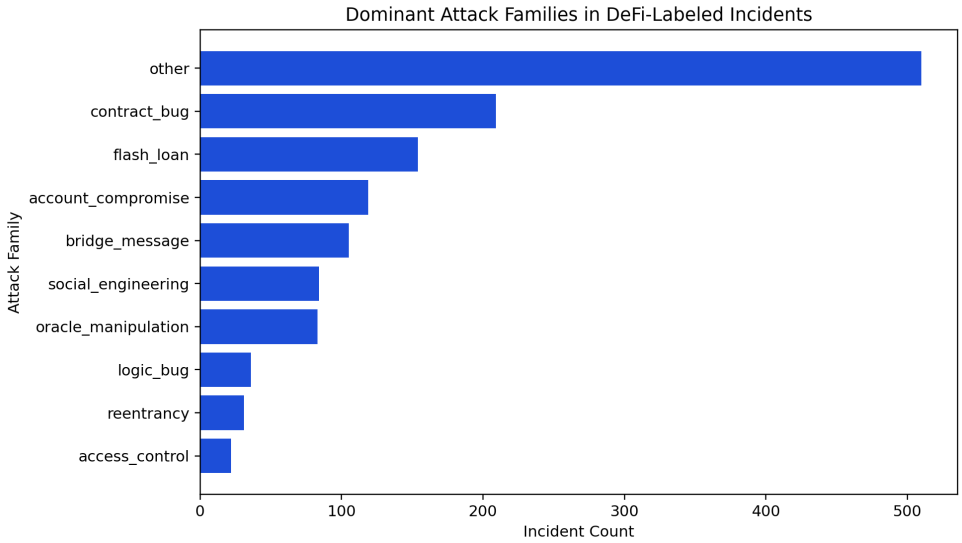

Figure 3: Dominant attack families in the DeFi-labeled corpus. The large residual `other` category shows why incident counts cannot be treated as a clean exploitability proxy.

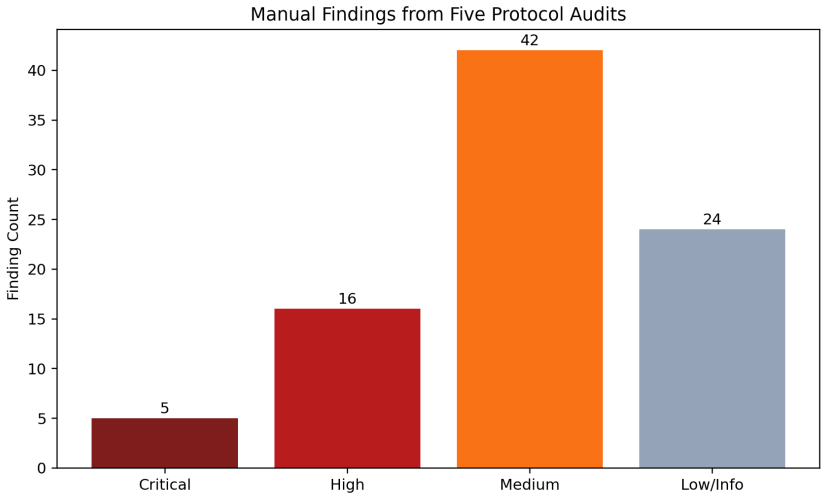

Figure 4: Manually triaged findings across the five deep protocol audits. Scanner output was much larger, but paper claims are based on the triaged set.

integration-dependent, and only 1 was confirmed exploitable with high confidence. Table 3 makes these categories explicit. The confirmed case is Morpho Blue, where the repository contains an already validated malicious-oracle market proof of concept. The conditional set consists of Pendle, Curve, and BENQI, each of which exposes a real mechanism family but still depends on downstream integrations, empty-market states, or runtime constraints before it becomes a clean exploit.

The protocol evidence therefore corrects a common intuition. Broad incident feeds make DeFi appear uniformly fragile, whereas protocol-level verification shows concentration: a small number of mechanism families continue to matter, but many high-TVL systems are protected by strong oracle design, constrained architecture, hardened governance, or simply the absence of a direct price-dependent lending path. The result is consistent with the intuition behind robust external price feeds such as Chainlink (Chainlink Labs, 2026), but our contribution is methodological rather

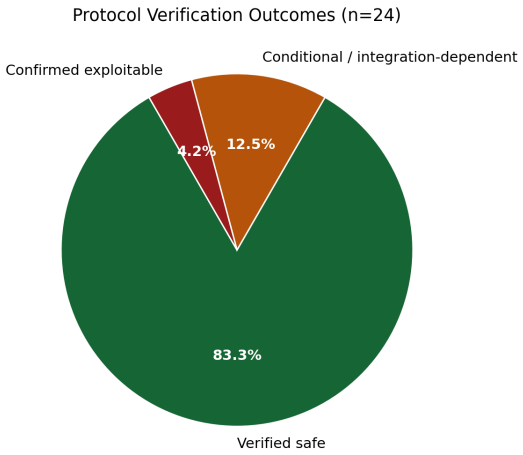

Figure 5: Outcome categories from the 24-protocol pure on-chain verification study. The high-confidence exploitable set is materially smaller than a naive reading of the incident corpus would suggest.

Table 3: Protocol-study outcomes under the repository's pure on-chain threat model.

Verdict	**Interpretation**	**Count**
Verified safe	No pure on-chain exploit path identified under the study assumptions	20
Conditional / integration-dependent	Attack surface exists, but exploitation depends on downstream integration, market state, or additional runtime assumptions	3
Confirmed exploitable	High-confidence pure on-chain exploit path supported by repository artifacts	1

than promotional: the repository demonstrates how to convert that intuition into explicit outcome categories.

Expressed as ratios, the result is even clearer. The protocol exposure ratio is approximately $4/24 = 16.7\%$ if conditional and confirmed cases are grouped together, and only $1/24 = 4.2\%$ if the threshold is raised to high-confidence confirmed exploitability. Those two values provide a practical upper and lower bound for how aggressively the current artifact set should be interpreted.

4.3 Replay Batch Results

Replay engineering is the final and narrowest evidence layer. The current batch contains twelve shortlisted incidents selected by loss magnitude, exploit-family relevance, and public reproducibility signals. Of those twelve, concrete replay logic has been implemented for three incidents: Euler Finance, BonqDAO and AllianceBlock, and Lendf.Me. Figure 6 places those cases alongside UwU Lend, which remains shortlisted but deferred because the exploit path is materially larger and requires a dedicated pass.

These cases are useful precisely because they represent different exploit families. Euler is a flash-loan-assisted liquidation and donation path with logic-level amplification, consistent with post-mortem analyses of the incident (Euler Labs, 2023). BonqDAO demonstrates how a manipulated reporting path can collapse collateral assumptions and enable outsized borrowing (The Block, 2023). Lendf.Me remains the canonical callback-driven reentrancy case in which ERC777 token hooks violate assumptions about call order and state finality (ZDNet, 2020). Together, the three cases support a narrower but stronger claim than the incident corpus alone: replayable exploitability in the reposi-

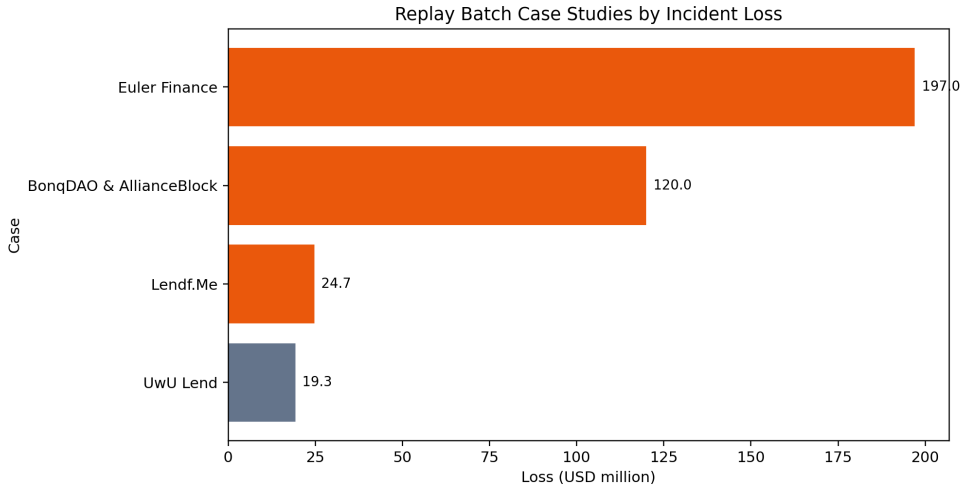

Figure 6: Replay batch case studies by reported loss. Orange bars indicate incidents with concrete replay logic already encoded in Foundry tests.

Table 4: Status of the principal replay case studies in the current paper.

Case	Family	Reported loss	Status	Fork-verified
Euler Finance	Flash-loan-assisted logic abuse	$197.0M	Replay logic implemented	No
BonqDAO & AllianceBlock	Oracle manipulation	$120.0M	Replay logic implemented	No
Lendf.Me	Reentrancy	$24.7M	Replay logic implemented	No
UwU Lend	Oracle manipulation	$19.3M	Shortlisted, deferred	No

tory clusters around a small set of mechanism families with well-understood control-flow or oracle preconditions.

The replay conversion rate is currently $3/12 = 25\%$. This number should not be interpreted as a failure. It is evidence that exploit reproduction is the most demanding stage in the workflow. Some incidents lack clean public exploit paths. Some depend on archive RPC state that is unavailable in the current shell configuration. Others, like UwU Lend, are multi-stage enough that forcing them into the batch would reduce correctness. A stronger paper is built by marking those boundaries explicitly rather than pretending that every high-loss incident is equally replayable.

5 Discussion

The results support a disciplined reinterpretation of DeFi security evidence. The incident layer says that DeFi losses are numerous, recurrent, and diverse. The protocol layer says that pure on-chain exploitability among major protocols is much narrower. The replay layer says that even historically important incidents do not all translate cleanly into executable proofs without substantial engineering and runtime context. The contribution of the workflow is therefore epistemic as much as empirical: it forces each claim to be attached to the evidence layer that actually supports it.

Three mechanism families dominate the current artifact set. Oracle manipulation remains the clearest bridge between protocol design assumptions and replayable exploitation, especially when external price-reporting paths or thin-liquidity dependencies interact with permissive borrowing logic. Flash-loan-assisted logic abuse remains important because it converts otherwise awkward state transitions into atomic capital-efficient strategies. Reentrancy remains relevant not because every protocol is equally vulnerable, but because callback semantics and external control flow still create sharp discontinuities in otherwise mature systems. These families are consistent with prior DeFi security literature (Qin et al., 2021; Zhou et al., 2023), but the repository adds concrete workflow scaffolding around them.

The audit and verification layers also suggest a practical distinction between *security debt* and *replayable exploitability*. A mature protocol may still accumulate dozens of medium-severity hardening issues or architectural concerns without exposing a high-confidence pure on-chain exploit path. This is visible in the five deep audits, where 87 manually triaged findings coexist with a protocol-verification result in which only one out of 24 major protocols is classified as confirmed exploitable. Research papers that ignore this distinction risk confusing engineering debt with imminent exploitability. Conversely, a workflow that only studies confirmed exploits may ignore the long tail of design weaknesses that shape future incidents.

The main limitation of the present paper is that the replay layer is still incomplete. The three principal replay cases have concrete logic encoded, but archive RPC validation has not been completed in the current environment. The protocol-verification layer also contains conditional cases whose exploitability depends on downstream integrations or edge-case market conditions. These limits do not invalidate the paper's argument; they define its scope. The paper is strongest as a methodology and intermediate empirical study, not yet as a definitive census of fully reproduced DeFi exploits.

That limitation points directly to the next phase of work. The repository should expand the replay suite from three implemented cases to at least five or eight archive-validated cases spanning oracle manipulation, reentrancy, logic abuse, and governance-assisted paths. It should also improve linkage between incident cards, protocol studies, and replay prerequisites so that each paper claim can be traced to a stable artifact. The refactored writing system created here is intended to make that next step cheaper, because it separates experiment generation, paper drafting, and revision rather than collapsing everything into a single ad hoc document.

6 Threats to Validity

Construct validity. The incident taxonomy is normalized from a public archive rather than derived from a single canonical benchmark. This creates unavoidable label noise, especially for records in which the public writeup mixes direct protocol failure with ancillary compromise or vague root-cause descriptions. We mitigate this by treating the incident corpus as a screening layer rather than final exploit evidence. Even so, the residual `other` category remains large, and this limits any fine-grained statistical claim about exact family prevalence.

Internal validity. The protocol-verification results rely on repository reports that already embed analyst judgment about exploitability under a narrow pure on-chain threat model. That judgment is deliberate, but it means the study favors precision over recall. Some incidents that were real in practice may be excluded because they required off-chain compromise, exceptional governance leverage, or market conditions not allowed by the study. Our conclusions should therefore be read as claims about replayable pure on-chain exploitability, not about total ecosystem risk.

External validity. The repository covers a non-trivial but still bounded protocol set. Five protocols received deep audits and twenty-four protocols were included in the pure on-chain verification study. These protocols are meaningful because they include major lending, AMM, and staking systems, but they do not represent every DeFi design. Results may not generalize cleanly to long-tail deployments, bespoke bridge architectures, or newly deployed protocols with immature code and low liquidity.

Artifact validity. Replay engineering remains an intermediate result rather than a final artifact evaluation. Three replay cases contain concrete logic in Foundry tests, but archive RPC execution was not completed in the present shell configuration. This matters because reproduction difficulty is part of the paper's argument. The artifact set can already support methodological conclusions, but a stronger paper will require archive-validated execution logs for at least a subset of the replay cases.

7 Artifact Availability and Reproducibility

The repository now exposes the paper workflow as a first-class artifact. The `idea/` directory records research questions and scope controls. The `exp/` directory contains the script that regenerates all current paper tables and figures from repository data, plus the derived CSV results used by

the draft. The `writing/` directory contains a blueprint, sectioned LaTeX sources, bibliography, and generated figures. The `revision/` directory records the weaknesses of the previous draft and the concrete changes made in this refactor. The `build/` directory exports the compiled PDF and build log.

Reproducing the current paper requires two commands from the paper root: first run the artifact script to regenerate result tables and figures, then run the PDF build script. Because the current draft relies only on local repository data and the already-generated bibliography, this workflow is deterministic under the existing environment. Replay validation is deliberately separated from paper building. That separation keeps the writing pipeline reproducible even when archive RPC credentials are absent, while also making clear that exploit replay claims remain bounded by the available runtime evidence.

8 Conclusion

This paper reorganizes the repository's security work into a research pipeline that begins with public incidents and ends with replay-oriented exploit claims. The central empirical result is straightforward: raw incident volume is much broader than confirmed pure on-chain exploitability. Within the current artifact set, the normalized corpus contains 1,378 DeFi-labeled incidents, but the 24-protocol verification study identifies only one high-confidence confirmed pure on-chain exploit path and three additional conditional cases. Replay engineering narrows the lens further, with concrete logic currently implemented for three representative historical exploits.

The paper's stronger contribution is methodological. By separating incident evidence, protocol evidence, and replay evidence, the refactored workflow makes it easier to write papers that are both richer and more honest. It encourages stronger claims where the evidence is strong, and more restrained language where the repository still contains only a scaffold, a shortlist, or a conditional finding. That is a better fit for DeFi security research than either a flat incident catalog or a scanner-driven vulnerability count.

The immediate next milestone is to convert the replay layer from implemented logic into archive-validated results. Once that happens for a broader set of cases, the same paper system can support a substantially stronger version of this study with deeper appendices, more complete case tables, and sharper comparative claims.

A Supplementary Tables

Table 5: Top-loss entries in the DeFi-labeled subset. The table illustrates why incident magnitude alone is a poor proxy for replayable contract exploitability.

Date	Target	Family	Loss
2019-06-29	PlusToken	other	$2.90B
2021-04-21	Thodex	other	$2.50B
2021-08-21	BitConnect	other	$2.00B
2025-02-21	Bybit	supply_chain	$1.46B
2019-10-08	WOTOKEN	other	$1.11B
2018-04-22	BeautyChain	other	$1.00B
2016-08-03	Bitfinex	other	$0.90B
2019-04-26	Bitfinex	other	$0.85B
2020-07-10	BitClub	other	$0.72B
2021-08-10	Poly Network	bridge_message	$0.61B

References

Chainlink Labs. Chainlink data feeds. `https://docs.chain.link/data-feeds`, 2026. Accessed 2026-03-31.

Table 6: Replay prerequisites for the three implemented case studies.

Case	Chain	Fork anchor	Test file
Euler Finance	Ethereum	block 16,817,995	`test/replay/EulerFinanceReplay.t.sol`
BonqDAO & AllianceBlock	Polygon	block 38,792,977	`test/replay/BonqDAOReplay.t.sol`
Lendf.Me	Ethereum	block 9,899,725	`test/replay/LendfMeReplay.t.sol`

Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges. In *IEEE Symposium on Security and Privacy*, 2020.

Euler Labs. Euler finance exploit post-mortem. `https://www.euler.finance/blog/euler-finance-exploit-post-mortem`, 2023.

Kaihua Qin, Liyi Zhou, Benjamin Livshits, and Arthur Gervais. Attacking the defi ecosystem with flash loans for fun and profit. *arXiv preprint arXiv:2003.03810*, 2021.

SlowMist. Hacked archive. `https://hacked.slowmist.io/`, 2026. Accessed 2026-03-31.

The Block. Bonqdao exploited for 88 million; allianceblock tokens stolen during the exploit. `https://www.theblock.co/post/207799/bonqdao-exploited-for-88-million-allianceblock-tokens-stolen-during-the-exploit`, 2023.

Sam M. Werner, Daniel Perez, Lewis Gudgeon, Ariah Klages-Mundt, Dominik Harz, and William J. Knottenbelt. Sok: Decentralized finance (defi). *arXiv preprint arXiv:2101.08778*, 2021.

ZDNet. Hackers steal 25 million worth of cryptocurrency from uniswap and lendf.me. `https://www.zdnet.com/article/hackers-steal-25-million-worth-of-cryptocurrency-from-uniswap-and-lendf-me/`, 2020.

Liyi Zhou, Xuxian Xiong, Jens Ernstberger, Stefanos Chaliasos, Zhipeng Wang, William Knottenbelt, Arthur Gervais, and Kaihua Qin. Sok: Decentralized finance security. *arXiv preprint arXiv:2301.10284*, 2023.